

Lecture 3 (9/18)

Sampling and Simulation

IID samples and empirical frequency

An **IID** sample $X_1, \dots, X_N$ consists of independent draws from the same distribution. If outcome x occurs $N(x)$ times,

$$P(X = x) \approx \frac{N(x)}{N},$$

and this approximation improves as N grows (discussed more in Unit 3).

IID samples and empirical frequency

An **IID** sample $X_1, \dots, X_N$ consists of independent draws from the same distribution. If outcome x occurs $N(x)$ times,

$$P(X = x) \approx \frac{N(x)}{N},$$

and this approximation improves as N grows (discussed more in Unit 3).

- A simulation is a means of generating samples from a computer.
- We should always try to generate simulated data that has similar properties as real data to test statistical methods.

Python basics: NumPy arrays

```
import numpy as np # a "wrapper" for Python lists
```

```
x = np.array([3, 1, 4, 1, 5])
```

```
y = np.array([1, 9, 4, 1, 5])
```

```
z = np.zeros(5) # np.array([0,0,0,0,0])
```

```
x[0] # 3
```

```
x[1:3] # [1, 4]
```

```
z[3] = x[1] # now z = np.array([0,0,0,1,0])
```

```
# boolean indexing
```

```
x > 2 # np.array([True,False,True,False,True])
```

```
x[x > 2] # [3, 4, 5]
```

```
x[y <= 1] # [3, 1]
```

```
x[(x == 1) & (y == 1)] # [1]
```

Practice: NumPy indexing

Example

```
x = np.array([3, 1, 4, 1, 5])
```

What does each of these evaluate to? (i) `x[2]` (ii) `x[0:2]` (iii) `x > 3` (iv) `x[x > 3]`

Practice: NumPy indexing

Example

```
x = np.array([3, 1, 4, 1, 5])
```

What does each of these evaluate to? (i) `x[2]` (ii) `x[0:2]` (iii) `x > 3` (iv) `x[x > 3]`

Example

```
x = np.array([3, 1, 4, 1, 5])  
y = np.array([1, 9, 4, 1, 5])
```

Write an expression that selects the entries of `x` where the corresponding entry of `y` is greater than 3.

Practice: NumPy indexing

Example

```
x = np.array([3, 1, 4, 1, 5])
```

What does each of these evaluate to? (i) `x[2]` (ii) `x[0:2]` (iii) `x > 3` (iv) `x[x > 3]`

Example

```
x = np.array([3, 1, 4, 1, 5])  
y = np.array([1, 9, 4, 1, 5])
```

Write an expression that selects the entries of `x` where the corresponding entry of `y` is greater than 3.

Example

Suppose `z = np.zeros(5)`. Write one line of code that sets `z[i]` equal to 1 wherever `x[i] == y[i]`.

Python basics: NumPy arrays

```
import numpy as np
```

```
x = np.array([3, 1, 4, 1, 5])
```

```
y = np.array([1, 9, 4, 1, 5])
```

```
len(x)          # 5
```

```
np.sum(x)       # 14
```

```
np.mean(x)      # 2.8
```

```
# elementwise
```

```
x + y          # [4, 10, 8, 2, 10]
```

```
x * y          # [3, 9, 16, 1, 25]
```

- `np.mean(x)` equals `np.sum(x) / len(x)`.
- Elementwise operations like `x + y` and `x * y` require arrays of the same length.

Practice: NumPy arrays and probability

Example

```
x = np.array([2, 5, 5, 7, 9])
```

What does each of these print? (i) `x[2]` (ii) `x[x >= 5]` (iii) `np.mean(x == 5)` and (iv) `len(x[x > 5])`.

Practice: NumPy arrays and probability

Example

```
x = np.array([2, 5, 5, 7, 9])
```

What does each of these print? (i) `x[2]` (ii) `x[x >= 5]` (iii) `np.mean(x == 5)` and (iv) `len(x[x > 5])`.

Example

Suppose `x` is a very long numpy array of samples of a random variable. Compute $P(X > 2)$ and $P(X = 4 \mid X > 2)$.

Practice: NumPy arrays and probability

Example

```
x = np.array([2, 5, 5, 7, 9])
```

What does each of these print? (i) `x[2]` (ii) `x[x >= 5]` (iii) `np.mean(x == 5)` and (iv) `len(x[x > 5])`.

Example

Suppose `x` is a very long numpy array of samples of a random variable. Compute $P(X > 2)$ and $P(X = 4 \mid X > 2)$.

Example

Suppose `x` and `y` are numpy arrays of samples (x_i, y_i) of (X, Y) , each taking values in $\{0, 1\}$. How would you compute (i) the marginal probability $P(Y = 1)$, (ii) the joint probability $P(X = 1, Y = 0)$, and (iii) the conditional probability $P(X = 0 \mid Y = 1)$?

Python basics: Generating samples

`np.random.choice(vals, p=p, size=N)` draws N iid samples from `vals`, using probabilities `p`.

```
vals = [0, 1, 2]      # sample space
p = [0.2, 0.3, 0.5]   # P(vals[i])
```

```
# N draws from vals
```

```
x = np.random.choice(vals, p=p, size=1000)
```

```
x == 2                # boolean array
```

```
np.mean(x == 2)       # estimates  $P(X = 2)$ 
```

```
# N draws from a Bernoulli(0.5)
```

```
y = np.random.choice([0, 1], p=[0.5, 0.5], size=1000)
```

```
np.mean(y)            # estimates  $P(Y = 1)$ 
```

Practice: simulating a conditional model

Example

Suppose x and y are generated by:

```
x = np.random.choice([0, 1], p=[0.5, 0.5], size=10000)
p0, p1 = [0.8, 0.2], [0.3, 0.7]    # dist of Y|X=0, Y|X=1
n0, n1 = np.sum(x == 0), np.sum(x == 1)
```

```
y = np.zeros(10000, dtype=int)
y[x == 0] = np.random.choice([0, 1], p=p0, size=n0)
y[x == 1] = np.random.choice([0, 1], p=p1, size=n1)
```

Write down the distribution of random variables X and $Y|X$ which are consistent with this code

Practice: simulating a conditional model

Example

Suppose x and y are generated by:

```
x = np.random.choice([0, 1], p=[0.5, 0.5], size=10000)
p0, p1 = [0.8, 0.2], [0.3, 0.7]    # dist of Y|X=0, Y|X=1
n0, n1 = np.sum(x == 0), np.sum(x == 1)
```

```
y = np.zeros(10000, dtype=int)
y[x == 0] = np.random.choice([0, 1], p=p0, size=n0)
y[x == 1] = np.random.choice([0, 1], p=p1, size=n1)
```

Write down the distribution of random variables X and $Y|X$ which are consistent with this code

Example

Let $X \sim \text{Bernoulli}(1/2)$ and $Y | X \sim \text{Bernoulli}(0.2 + 0.5X)$. Make Python arrays x and y which contain samples of these.

Control

```
# if / else
x = 7
if x > 5:
    label = "big"
else:
    label = "small"

# for loop
total = 0
for i in [1, 2, 3, 4]:
    total = total + i
# total = 10

# inline for loop (list comprehension)
squares = [i**2 for i in range(5)]
# squares = [0, 1, 4, 9, 16]
```

Practice: control flow

Example

```
x = np.array([2, 5, 1, 8, 3])  
total = 0  
for v in x[x > 2]:  
    total = total + v
```

What is total equal to after this loop runs?

Practice: control flow

Example

```
x = np.array([2, 5, 1, 8, 3])
total = 0
for v in x[x > 2]:
    total = total + v
```

What is total equal to after this loop runs?

Example

```
x = np.array([4, -2, 7, -5, 3])
count = 0
for v in x:
    if v < 0:
        count = count + 1
```

What is count equal to after this loop runs?

Practice: control flow

Example

```
x = np.array([2, 5, 1, 8, 3])
total = 0
for v in x[x > 2]:
    total = total + v
```

What is total equal to after this loop runs?

Example

```
x = np.array([4, -2, 7, -5, 3])
count = 0
for v in x:
    if v < 0:
        count = count + 1
```

What is count equal to after this loop runs?

Example

Write a list comprehension that produces [1, 4, 9, 16, 25] using range(1, 6).

Python basics: functions

`def` defines a reusable block of code that takes inputs (*arguments*) and can return an output.

```
def estimate_prob(x, value):  
    return np.mean(x == value)
```

```
x = np.array([0, 1, 1, 0, 1])  
estimate_prob(x, 1)    # 0.6  
estimate_prob(x, 0)    # 0.4
```

- `x` and `value` are just local names for whatever gets passed in when the function is called.

Example

Write code to generate samples from the following distribution

$P(C, M)$	$M = 1$	$M = 0$
$C = 1$ (diagnosed)	0.0021	0.0019
$C = 0$ (not diagnosed)	0.4899	0.5061

Write two functions, one that generates samples (C, M) from the joint distribution and another that generates samples (C, M) each independently from the marginal distributions. Then for different sample sizes compare the resulting 2×2 table of frequencies.

Practice: simulating the Markov chain

Example

Recall the Markov chain from Lecture 2:

$$X_0 \sim \text{Bernoulli}(1/2), \quad X_{t+1} \mid X_t \sim \text{Bernoulli}(q_1 X_t + q_0(1 - X_t)).$$

Write code to simulate $X_0, \dots, X_{T-1}$, then estimate $\lim_{t \rightarrow \infty} P(X_t = 1)$ once the chain has settled down.

Practice: simulating the Markov chain

Example

Recall the Markov chain from Lecture 2:

$$X_0 \sim \text{Bernoulli}(1/2), \quad X_{t+1} \mid X_t \sim \text{Bernoulli}(q_1 X_t + q_0(1 - X_t)).$$

Write code to simulate $X_0, \dots, X_{T-1}$, then estimate $\lim_{t \rightarrow \infty} P(X_t = 1)$ once the chain has settled down.

```
q0, q1 = 0.2, 0.7
T = 1000
x = np.zeros(T, dtype=int)
x[0] = np.random.choice([0, 1], p=[0.5, 0.5])
for t in range(1, T):
    if x[t - 1] == 1:
        x[t] = np.random.choice([0, 1], p=[1 - q1, q1])
    else:
        x[t] = np.random.choice([0, 1], p=[1 - q0, q0])

np.mean(x[500:])    # should be close to q0 / (1 - q1 + q0)
```